

AMA 564 Deep Learning

2026 Spring

Lecture 12

30K Tasks

Step 1

**Collect demonstration data,
and train a supervised policy.**

A prompt is
sampled from our
prompt dataset.

Explain the moon
landing to a 6 year old

A labeler
demonstrates the
desired output
behavior.

Some people went
to the moon...

This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

A prompt and
several model
outputs are
sampled.

Explain the moon
landing to a 6 year old

A Explain gravity...

B Explain war...

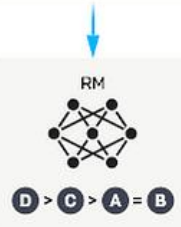
C Moon is natural
satellite of...

D People went to
the moon...

A labeler ranks
the outputs from
best to worst.

D > C > A = B

This data is used
to train our
reward model.



Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.

Write a story
about frogs

The policy
generates
an output.



Once upon a time...

The reward model
calculates a
reward for
the output.



The reward is
used to update
the policy
using PPO.

r_k

- How do we actually change our LM parameters θ to maximize this?

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)}[R(\hat{s})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{s} \sim p_{\theta_t}(s)}[R(\hat{s})]$$

How do we estimate
this expectation??

What if our reward
function is non-
differentiable??

- Policy gradient** methods in RL (e.g., REINFORCE; [[Williams, 1992](#)]) give us tools for estimating and optimizing this objective.

- We want to obtain (defn. of expectation) (linearity of gradient)

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

- Here we'll use a very handy trick known as the **log-derivative trick**. Let's try taking the gradient of $\log p_{\theta}(s)$

$$\nabla_{\theta} \log p_{\theta}(s) = \frac{1}{p_{\theta}(s)} \nabla_{\theta} p_{\theta}(s) \quad \Rightarrow \quad \nabla_{\theta} p_{\theta}(s) = p_{\theta}(s) \nabla_{\theta} \log p_{\theta}(s)$$

(chain rule)

- Plug back in:

$$\sum_s R(s) \nabla_{\theta} p_{\theta}(s) = \sum_s p_{\theta}(s) R(s) \nabla_{\theta} \log p_{\theta}(s)$$

This is an expectation of this

- Now we have put the gradient “inside” the expectation, we can approximate this objective with Monte Carlo samples:

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta} \log p_{\theta}(s_i)$$

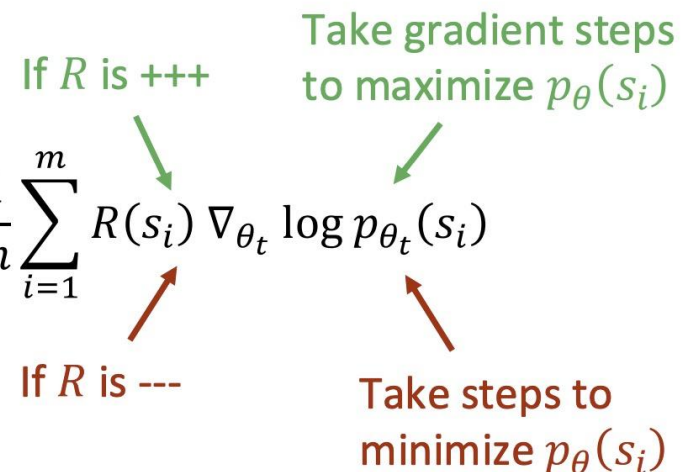
This is why it's called “**reinforcement learning**”: we **reinforce** good actions, increasing the chance they happen again.

- Giving us the update rule: $\theta_{t+1} := \theta_t + \alpha \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta_t} \log p_{\theta_t}(s_i)$

This is **heavily simplified**! There is a *lot* more needed to do RL w/ LMs. **Can you see any problems with this objective?**

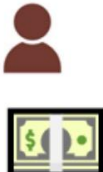
If R is +++
Take gradient steps to maximize $p_{\theta}(s_i)$

If R is ---
Take steps to minimize $p_{\theta}(s_i)$



- Awesome: now for any **arbitrary, non-differentiable reward function** $R(s)$, we can train our language model to maximize expected reward.
- Not so fast! (Why not?)
- **Problem 1:** human-in-the-loop is expensive!
 - **Solution:** instead of directly asking humans for preferences, **model their preferences** as a separate (NLP) problem! [[Knox and Stone, 2009](#)]

An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

$$R(s_1) = 8.0$$


The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.

$$R(s_2) = 1.2$$


Train an LM $RM_\phi(s)$ to
predict human
preferences from an
annotated dataset, then
optimize for RM_ϕ instead.

- **Problem 2:** human judgments are noisy and miscalibrated!
- **Solution:** instead of asking for direct ratings, ask for **pairwise comparisons**, which can be more reliable [[Phelps et al., 2015](#); [Clark et al., 2018](#)]

A 4.2 magnitude
earthquake hit
San Francisco,
resulting in
massive damage.

$$R(s_3) = \begin{matrix} s_3 \\ 4.1? & 6.6? & 3.2? \end{matrix}$$

- **Problem 2:** human judgments are noisy and miscalibrated!
- **Solution:** instead of asking for direct ratings, ask for **pairwise comparisons**, which can be more reliable [[Phelps et al., 2015](#); [Clark et al., 2018](#)]

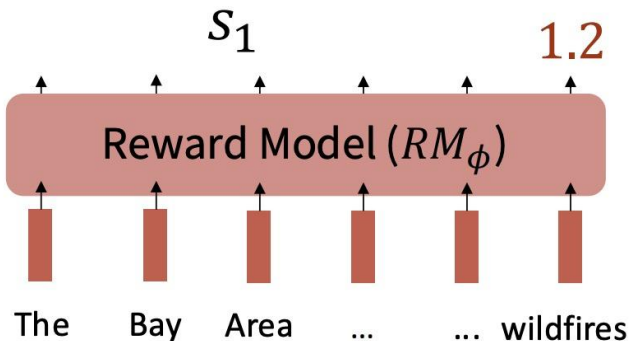
An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

>

A 4.2 magnitude
earthquake hit
San Francisco,
resulting in
massive damage.

>

The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.



S_3

Bradley-Terry [1952] paired comparison model

$$J_{RM}(\phi) = -\mathbb{E}_{(s^w, s^l) \sim D} [\log \sigma(RM_\phi(s^w) - RM_\phi(s^l))]$$

“winning”
sample

“losing”
sample

S_2

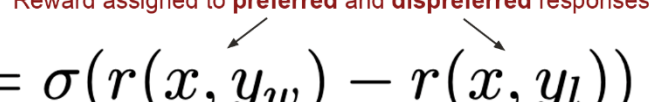
s^w should score
higher than s^l

Feedback comes as **preferences over model samples**: $\mathcal{D} = \{x^i, y_w^i, y_l^i\}$



Bradley-Terry Model connects rewards to preferences:

Reward assigned to **preferred** and **dispreferred** responses

$$p(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$


Train the reward model by **minimizing negative log likelihood**:

$$\mathcal{L}_R(\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

Now we have a **reward model** r_ϕ that represents* **goodness according to humans**

Now, learn a policy π_θ achieving **high reward** while **staying close** to original model π_{ref}

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)]$$

Sample from policy



Want high reward...



...but keep KL to original model small!



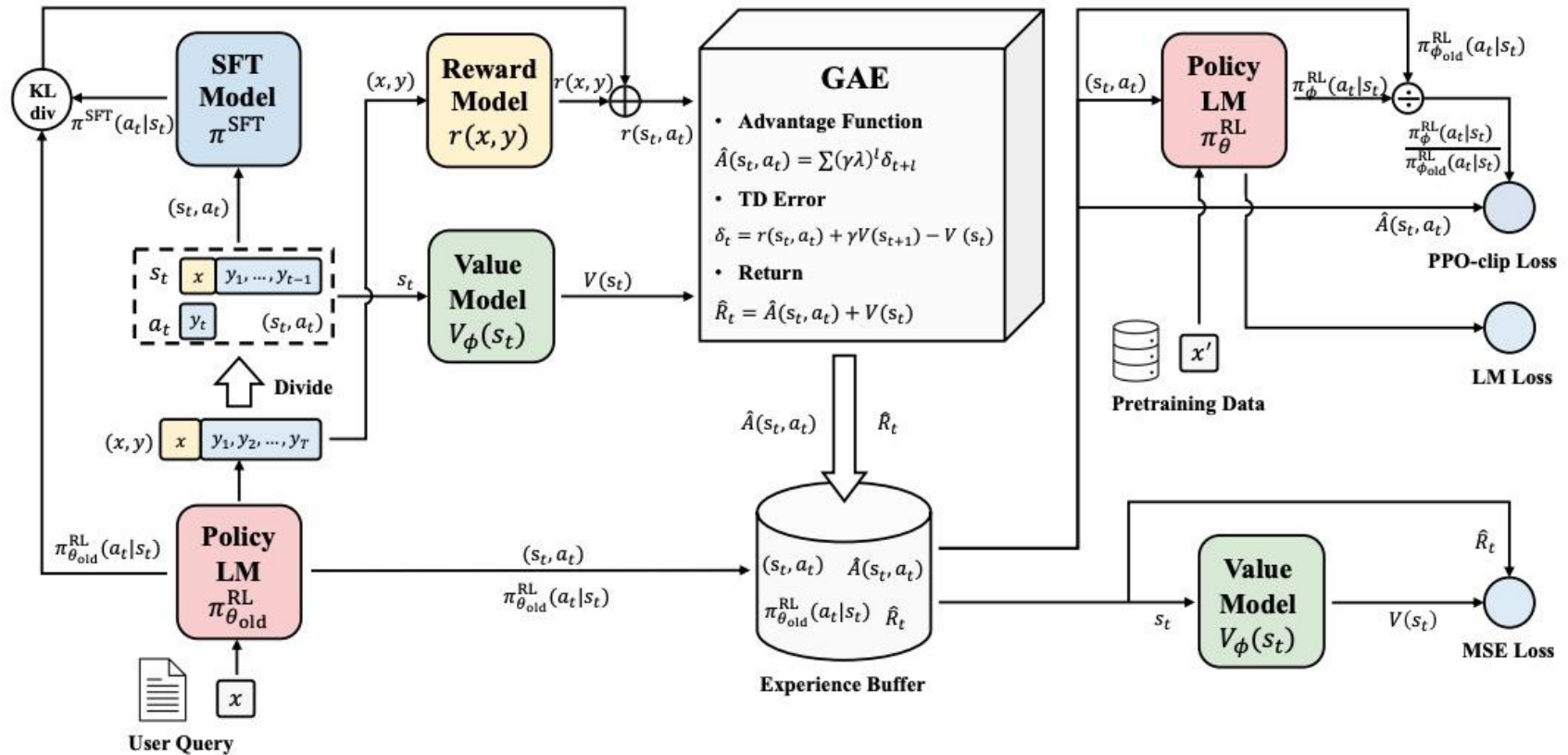


Figure 1: PPO workflow, depicting the sequential steps in the algorithm's execution. The process begins with sampling from the environment, followed by the application of GAE for improved advantage approximation. The diagram then illustrates the computation of various loss functions employed in PPO, signifying the iterative nature of the learning process and the policy updates derived from these losses.



- **Accelerate RLHF with Variance Increase**
- **Direct Preference Optimization**

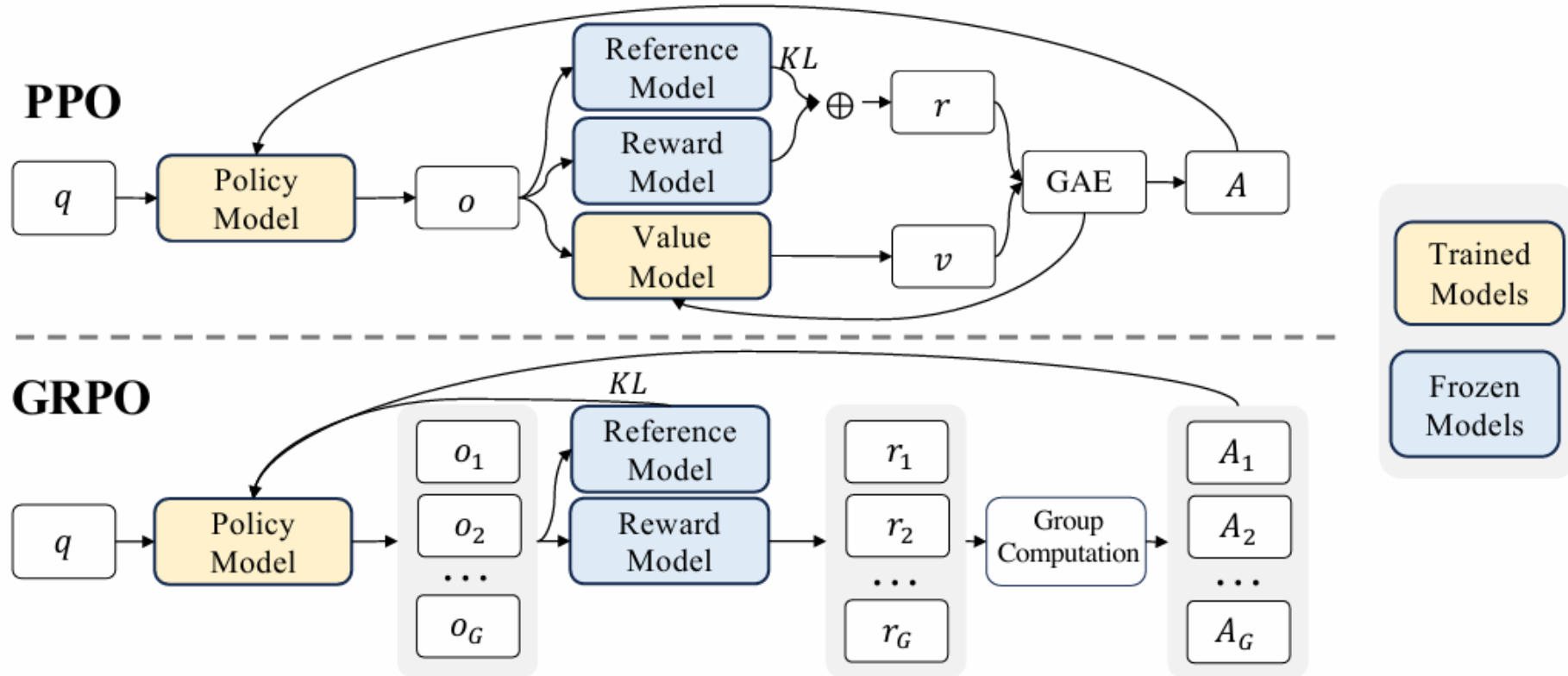


Fig. 3. Comparison of popular RL fine-tuning methods in RLHF

- GRPO has exhibited comparable performance with less computational cost compared to PPO.

[2] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., ... & Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

[3] Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., ... & He, Y. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

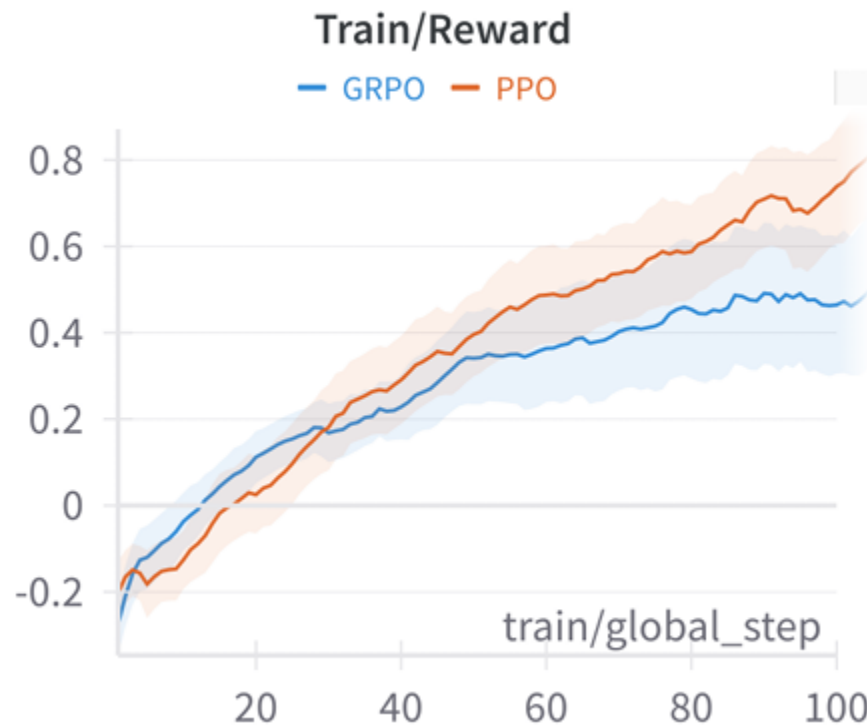


Fig. 4. Performance of PPO and GRPO during training process

- GRPO has comparable performance compared to PPO at the beginning but **suffers from slow training in the end.**

- GRPO has comparable performance compared to PPO at the beginning but **suffers from slow training in the end.**
- Although GRPO needs less training resources, it suffers from its slow training in practice.
- **How to accelerate GRPO?**

[2] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., ... & Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

[3] Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., ... & He, Y. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

[4] Hou, Z., Du, P., Niu, Y., Du, Z., Zeng, A., Liu, X., ... & Dong, Y. (2024). Does RLHF Scale? Exploring the Impacts From Data, Model, and Method. *arXiv preprint arXiv:2412.06000*.

[5] Li, G., Lin, M., Galanti, T., Tu, Z., & Yang, T. (2025). DisCO: Reinforcing Large Reasoning Models with Discriminative Constrained Optimization. *arXiv preprint arXiv:2505.12366*.

- Both **accuracy** and **reward variance** matter in RLHF training!

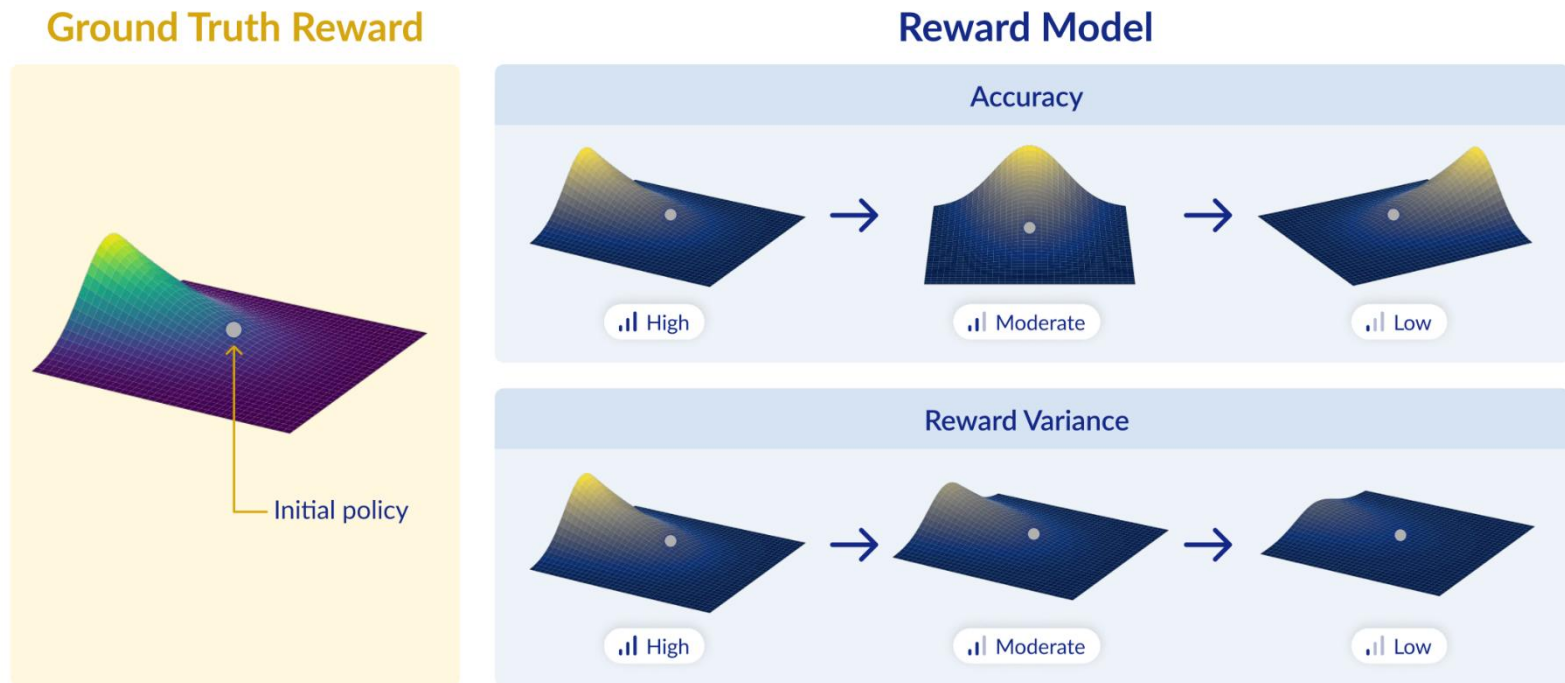


Fig. 5. Illustration of the roles of accuracy and reward variance in RLHF

Theorem. For RLHF objective

$$\mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\mathbb{E}_{\mathbf{y} \sim \pi_{\theta}(\cdot | \mathbf{y})} [r(\mathbf{x}, \mathbf{y})] - \lambda D_{\text{KL}}(\pi_{\theta}(\cdot | \mathbf{y}) || \pi_{\text{ref}}(\cdot | \mathbf{y})) \right]$$

Given any $\gamma > 0$, prompt $\mathbf{x} \in \mathcal{X}$, the time it takes until

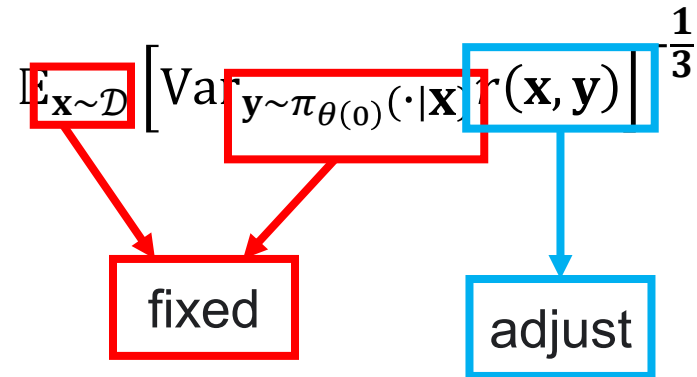
$$\mathbb{E}_{\mathbf{y} \sim \pi_{\theta(t)}(\cdot | \mathbf{y})} [r(\mathbf{x}, \mathbf{y})] \geq \mathbb{E}_{\mathbf{y} \sim \pi_{\theta(0)}(\cdot | \mathbf{y})} [r(\mathbf{x}, \mathbf{y})] + \gamma$$

is at least:

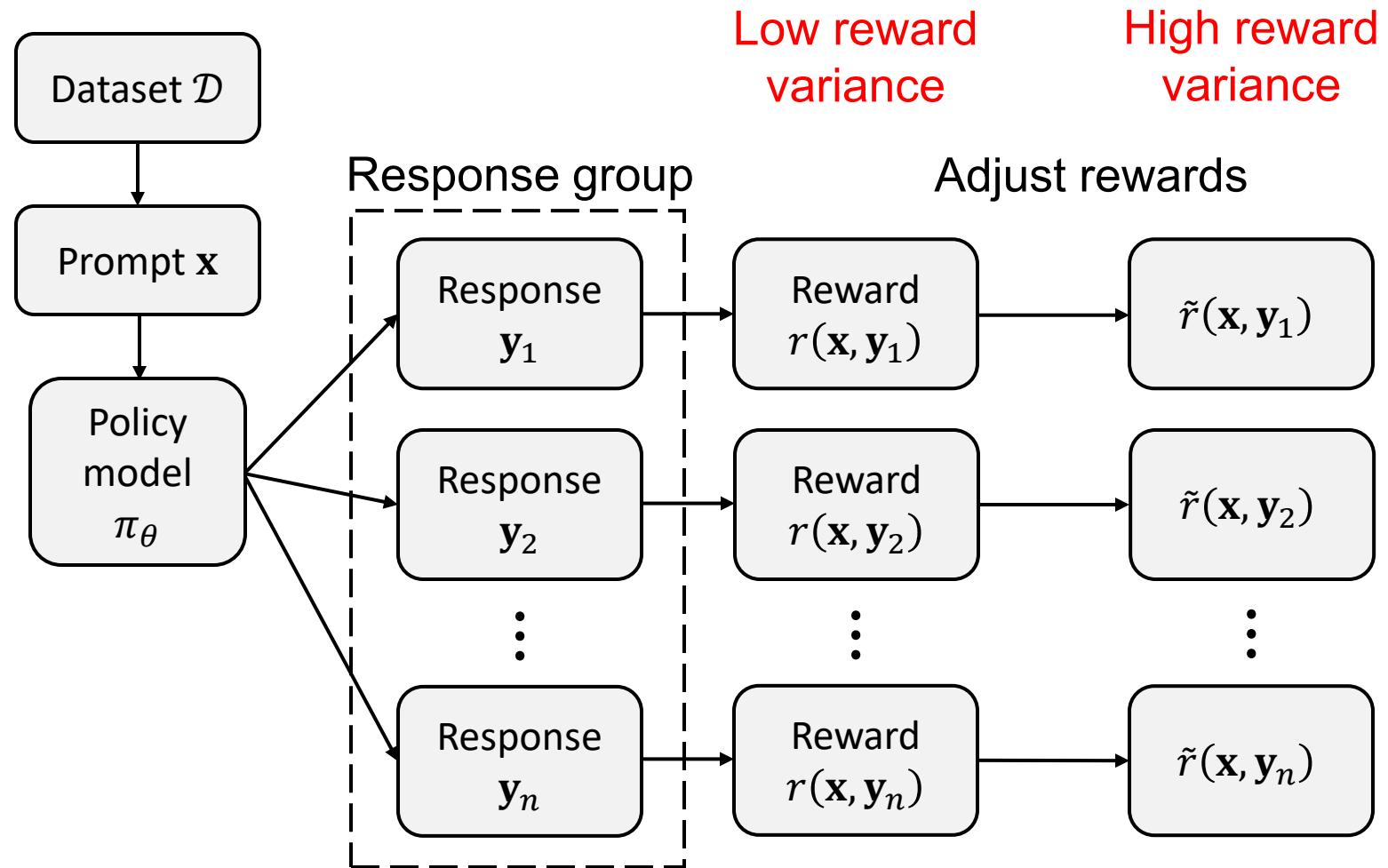
$$\Omega \left(\mathbb{E}_{\mathbf{x}' \sim \mathcal{D}} \left[\text{Var}_{\mathbf{y} \sim \pi_{\theta(0)}(\cdot | \mathbf{x}')} r(\mathbf{x}', \mathbf{y}) \right]^{-\frac{1}{3}} \right)$$

Natural idea: accelerating by increasing $\mathbb{E}_{\mathbf{x}' \sim \mathcal{D}} \left[\text{Var}_{\mathbf{y} \sim \pi_{\theta(0)}(\cdot | \mathbf{x}')} r(\mathbf{x}', \mathbf{y}) \right]^{-\frac{1}{3}}$

How to increase?



Idea: adjust $r(\mathbf{x}, \mathbf{y})$ to increase $\text{Var}_{\mathbf{y} \sim \pi_{\theta(0)}(\cdot | \mathbf{x})} r(\mathbf{x}, \mathbf{y})$ for each $\mathbf{x} \in \mathcal{D}$



Reward adjustment model

- Objective: increase reward variance.
- Requirement 1: preserve reward boundness.
- Requirement 2: preserve reward expectation.
- Requirement 3: preserve relative preferences.
- WLOG, suppose original reward model is $r: \mathcal{X} \times \mathcal{Y} \rightarrow [m, M]$ and $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n$ are ordered such that

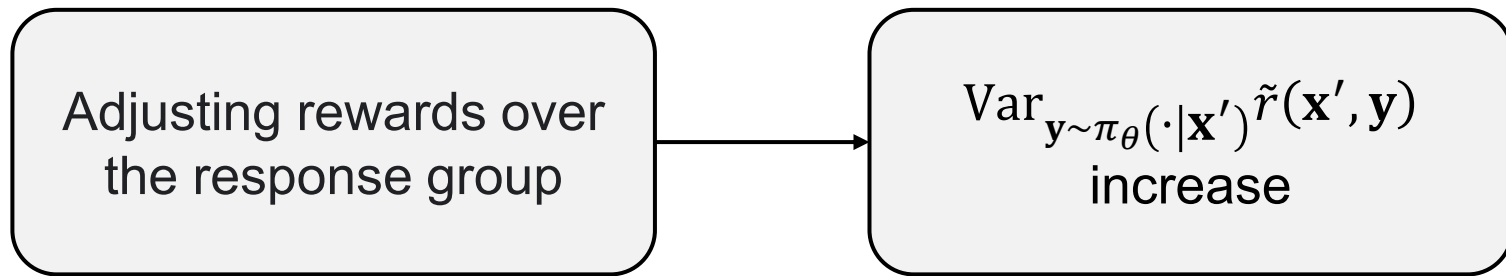
$$r(\mathbf{x}, \mathbf{y}_1) \geq r(\mathbf{x}, \mathbf{y}_2) \geq \dots \geq r(\mathbf{x}, \mathbf{y}_n).$$

- We propose a reward adjustment model.

$$\begin{aligned} \max_{\mathbf{z} \in \mathbb{R}^n} \quad & f(\mathbf{z}) := \sum_{i=1}^n p_i z_i^2 \\ \text{s. t.} \quad & m \leq z_i \leq M \quad \forall 1 \leq i \leq n, & \text{Boundedness} \\ & \sum_{i=1}^n p_i z_i = \sum_{i=1}^n p_i r_i, & \text{Reward expectation} \\ & z_i \geq z_{i+1} \quad \forall 1 \leq i \leq n-1, & \text{Relative preferences} \end{aligned}$$

where $p_i = \pi_{\theta(0)}(\mathbf{y}_i | \mathbf{x})$.

- Suppose we have a solution $\mathbf{z}^*$. Then the reward $r(\mathbf{x}, \mathbf{y}_i)$ is adjusted to $\tilde{r}(\mathbf{x}, \mathbf{y}_i) = \mathbf{z}_i^*$.



Theorem 1. Let $\mathbf{x} \in \mathcal{D}$ be a given prompt and $\{\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n\} \subset \mathcal{Y}$ be the response group. If the reward $(\tilde{r}(\mathbf{x}, \mathbf{y}_1), \tilde{r}(\mathbf{x}, \mathbf{y}_2), \dots, \tilde{r}(\mathbf{x}, \mathbf{y}_n))$ is a global optimal solution of the reward adjustment model, with $r_i = r(\mathbf{x}, \mathbf{y}_i)$ and $p_i = \pi_\theta(\mathbf{y}_i | \mathbf{x}) \forall i \in \{1, 2, \dots, n\}$, then the reward variance of policy model $\pi_\theta(\cdot | \mathbf{x})$ over the response space can be increased for the prompt $\mathbf{x} \in \mathcal{D}$.

Properties of the proposed reward adjustment model:

- Non-convex optimization problem, usually NP-hard to solve.
- The feasible set is a bounded and non-empty polyhedral.

$$\mathcal{P} := \{ \mathbf{z} \in \mathbb{R}^n \mid \sum_{i=1}^n p_i z_i = \sum_{i=1}^n p_i r_i, m \leq z_i \leq M \quad \forall 1 \leq i \leq n, z_i \geq z_{i+1} \quad \forall 1 \leq i \leq n-1 \}$$

- Objective function is convex.

- The global optimal solution is attained at one of the extreme points.

Lemma 1. Define a subset of extreme point set $\mathcal{V}$ as

$$\mathcal{V}^* := \left\{ \mathbf{v} \in \mathcal{V} \mid f(\mathbf{v}) = \max_{\mathbf{v}' \in \mathcal{V}} f(\mathbf{v}') \right\},$$

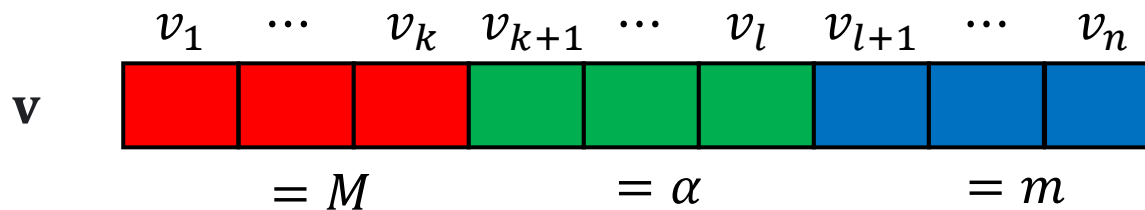
then the point(s) in $\mathcal{V}^*$ are global solution(s) to the reward adjustment model.

- The set of extreme points can be characterized.

Lemma 2. The set of extreme points $\mathcal{V}$ of $\mathcal{P}$ is

$$\mathcal{V} = \left\{ \mathbf{v} = (v_1, \dots, v_n) \in \mathcal{P} \left| \begin{array}{l} \exists 0 \leq k \leq l \leq n, m < \alpha < M \text{ s.t.} \\ v_1 = \dots = v_k = M, \\ v_{k+1} = \dots = v_l = \alpha, \\ v_{l+1} = \dots = v_n = m, \\ \alpha = \frac{\sum_{i=1}^n p_i r_i - M \sum_{i=1}^k p_i - m \sum_{i=l+1}^n p_i}{\sum_{i=k+1}^l p_i} \end{array} \right. \right\}$$

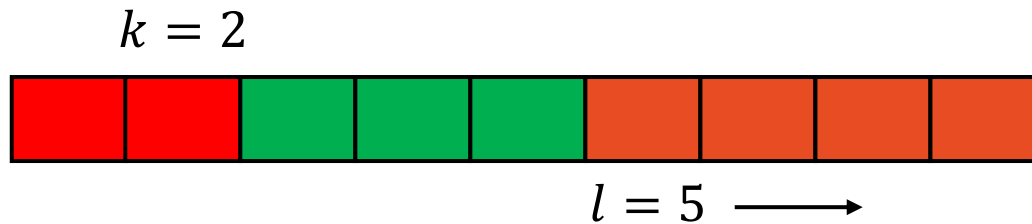
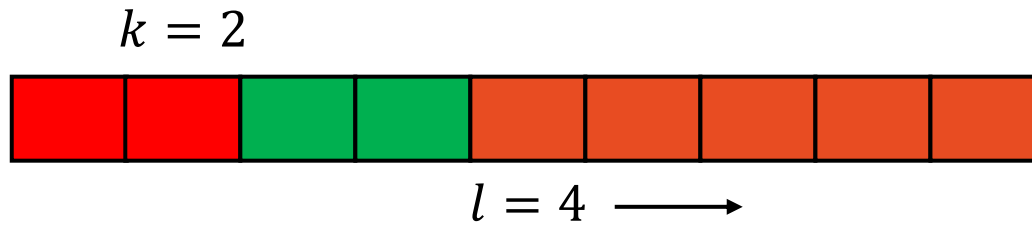
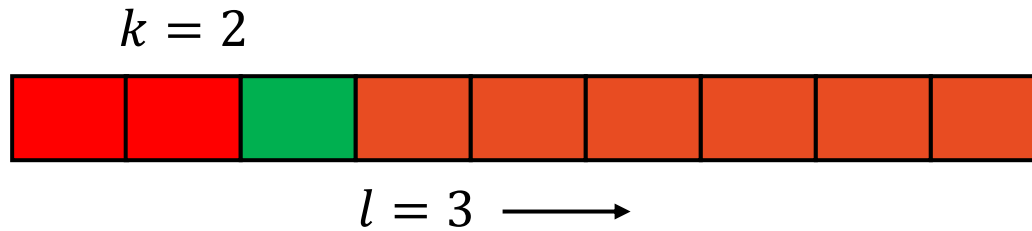
- Therefore, the extreme point $\mathbf{v} = (v_1, v_2, \dots, v_n)$ has the following form.



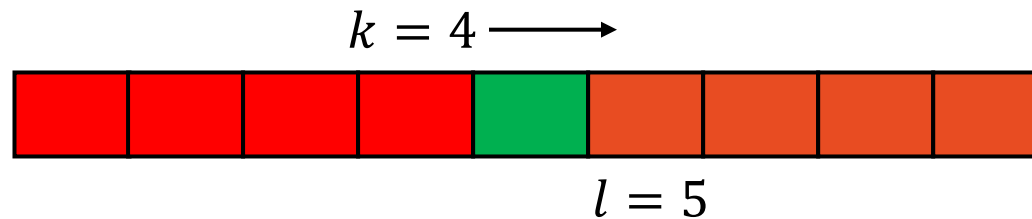
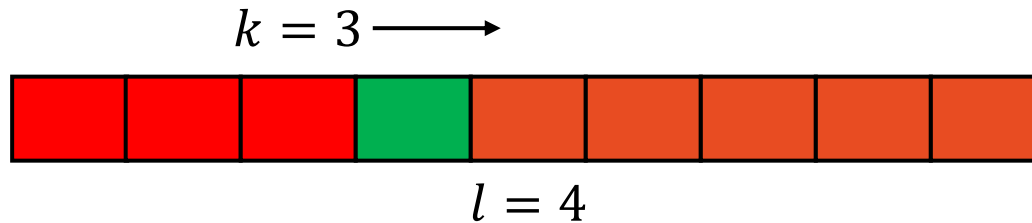
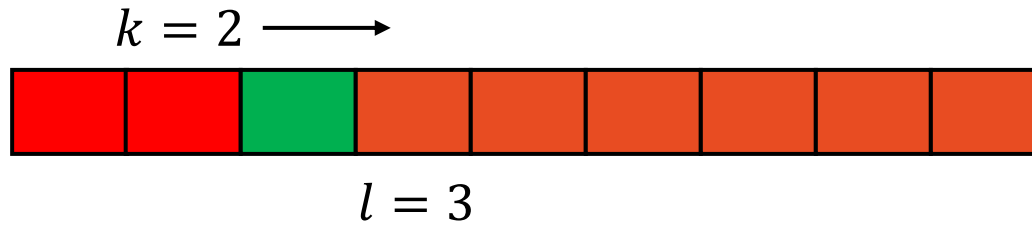
- Based on the special form, an $O(n)$ solving algorithm is designed.

Algorithms

- Enumerate l from k to n . (inner loop)



- Enumerate k from 0 to n . (outer loop)



- An $O(n^2)$ algorithm to find the global optimal solution.

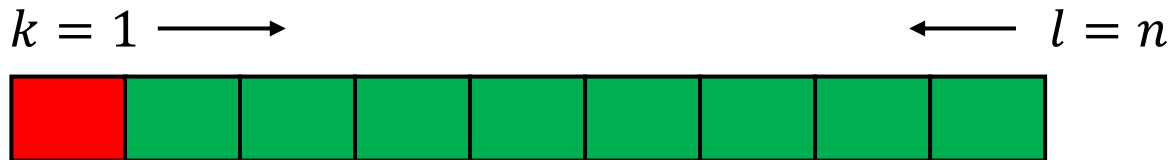
Algorithm 1 Enumeration search algorithm for (3)

```
1: Input:  $p = (p_1, p_2, \dots, p_n)$  with  $0 < p_i < 1$ . A sorted reward  $r = (r_1, r_2, \dots, r_n)$ . An upper bound  $M$  and a lower bound  $m$ .
2: Output: A optimal solution  $\mathbf{z}^*$  to (3) and the optimal objective value  $f^*$ .
3: Initialization:  $c = \sum_{i=1}^n p_i r_i$ ,  $\mathbf{z}^* = r$ ,  $f^* = 0$ ,  $k^* = 0$ ,  $l^* = 0$ ,  $\alpha^* = 0$ .
4: for  $k = 0, 1, \dots, n$  do
5:   for  $l = k, k + 1, \dots, n$  do
6:     Compute  $\alpha$  based on lemma 2.
7:     Compute current objective value  $\bar{f}$  based on  $c, M, m, p$ .
8:     if  $\bar{f} > f^*$  then
9:       Set  $f^* = \bar{f}$ ,  $k^* = k$ ,  $l^* = l$ ,  $\alpha^* = \alpha$ .
10:    end if
11:  end for
12: end for
13: Set  $z_1^* = \dots = z_{k^*}^* = M, z_{k^*+1}^* = \dots = z_{l^*}^* = \alpha^*, z_{l^*+1}^* = \dots = z_n^* = m$ .
14: return  $\mathbf{z}^* = (z_1^*, z_2^*, \dots, z_n^*)$  and  $f^*$ 
```

→ outer loop

→ inner loop

- An $O(n)$ search algorithm.
- Search strategy: start search from two ends to the center.



- Once a solution becomes infeasible, all subsequent solutions along that search direction are infeasible.

Define $S_A = \sum_{i=1}^k p_i$, $S_B = \sum_{i=k+1}^l p_i$, $S_C = \sum_{i=l+1}^n p_i$.

Lemma 3. Suppose $k < l - 1$

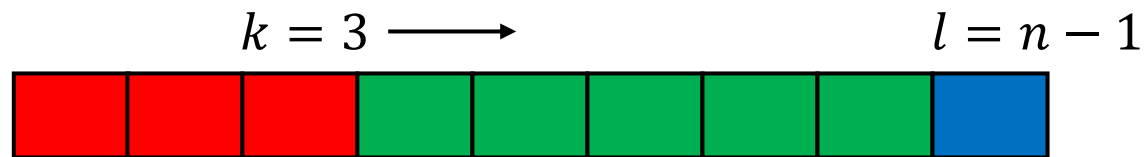
If $M < \frac{c - MS_A - mS_C}{S_B}$, then $\frac{c - MS_A - mS_C}{S_B} < \frac{c - M(S_A + p_{k+1}) - mS_C}{S_B - p_{k+1}}$;

If $m > \frac{c - MS_A - mS_C}{S_B}$, then $\frac{c - MS_A - mS_C}{S_B} > \frac{c - MS_A - m(S_C + p_l)}{S_B - p_l}$.

- Once a solution becomes infeasible, all subsequent solutions along that search direction are infeasible.
- Illustration:



If $k = 2, l = n - 1$ is infeasible,



then $k \geq 3, l = n - 1$ are all infeasible.

- The objective function value exhibits a monotonically increasing trend if the solution is feasible.

Lemma 5. Assume $m < \alpha < M$.

If $\frac{c - M(S_A + p_{k+1}) - mS_C}{S_B - p_{k+1}} > m$, then it can strictly increase the objective function value by setting $k = k + 1$ and $l = l$;

If $\frac{c - MS_A - m(S_C + p_l)}{S_B - p_l} < M$, then it can strictly increase the objective function value by setting $k = k$ and $l = l - 1$.

- The objective function value exhibits a monotonically increasing trend if the solution is feasible.
- Illustration



If those three solutions are all feasible, then $f_2 > f_1 > f_0$



One-pass search algorithm $O(n)$

- One-pass search algorithm is an $O(n)$ algorithm.
- If the rewards are not sorted at first, then it will require $O(n \log n)$ to sort the rewards and probabilities.

Algorithm 2 One-pass search algorithm for (3)

```

1: Input:  $p = (p_1, p_2, \dots, p_n)$  with  $0 < p_i < 1$ . A sorted reward  $r = (r_1, r_2, \dots, r_n)$ . An upper bound  $M$  and a lower bound  $m$ .
2: Output: A optimal solution  $\mathbf{z}^*$  to (3) and the optimal objective value  $f^*$ .
3: Initialization:  $c = \sum_{i=1}^n p_i r_i$ ,  $k = 0$ ,  $l = n$ ,  $iter = 0$ .
4: Define  $C(0) = 0$  and compute the cumulative sums  $C(i) = \sum_{j=1}^i p_j$   $1 \leq i \leq n$ .
5: while  $iter < n$  do
6:   Compute  $S_A = C(k)$ ,  $S_C = C(n) - C(l)$ ,  $S_B = C(l) - C(k)$ .
7:   if  $S_B \leq 0$  then
8:     Break.
9:   end if
10:  Set  $\alpha = (c - MS_A - mS_C)/S_B$ . Compute objective  $f = S_A M^2 + S_B \alpha^2 + S_C m^2$ .
11:  Set  $f^* = f$ ,  $k^* = k$ ,  $l^* = l$ ,  $flags = 0$ .
12:  if  $k < l$  and  $S_B - p_{k+1} > 0$  then
13:     $\bar{\alpha} = \frac{c - (S_A + p_{k+1})M - S_C m}{S_B - p_{k+1}}$ .
14:    if  $m \leq \bar{\alpha} \leq M$  then
15:       $f_{new} = (S_A + p_{k+1})M^2 + (S_B - p_{k+1})\bar{\alpha}^2 + S_C m^2$ .
16:      if  $f_{new} > f^*$  then
17:         $f^* = f_{new}$ ,  $k^* = k + 1$ ,  $l^* = l$ ,  $flags = 1$ .
18:      end if
19:    end if
20:  end if
21:  if  $l > k$  and  $S_B - p_l > 0$  then
22:     $\bar{\alpha} = \frac{c - S_A M - (S_C + p_l)m}{S_B - p_l}$ .
23:    if  $m \leq \bar{\alpha} \leq M$  then
24:       $f_{new} = S_A M^2 + (S_B - p_l)\bar{\alpha}^2 + (S_C + p_l)m^2$ .
25:      if  $f_{new} > f^*$  then
26:         $f^* = f_{new}$ ,  $k^* = k$ ,  $l^* = l - 1$ ,  $flags = 1$ .
27:      end if
28:    end if
29:  end if
30:  if  $flags = 1$  then
31:     $k = k^*$ ,  $l = l^*$ .
32:    Continue.
33:  else
34:    Break.
35:  end if
36:   $iter = iter + 1$ .
37: end while
38: Set  $z_1^* = \dots = z_{k^*}^* = M$ ,  $z_{k^*+1}^* = \dots = z_{l^*}^* = \alpha$ ,  $z_{l^*+1}^* = \dots = z_n^* = m$ .
39: return  $\mathbf{z}^* = (z_1^*, z_2^*, \dots, z_n^*)$  and  $f^*$ 

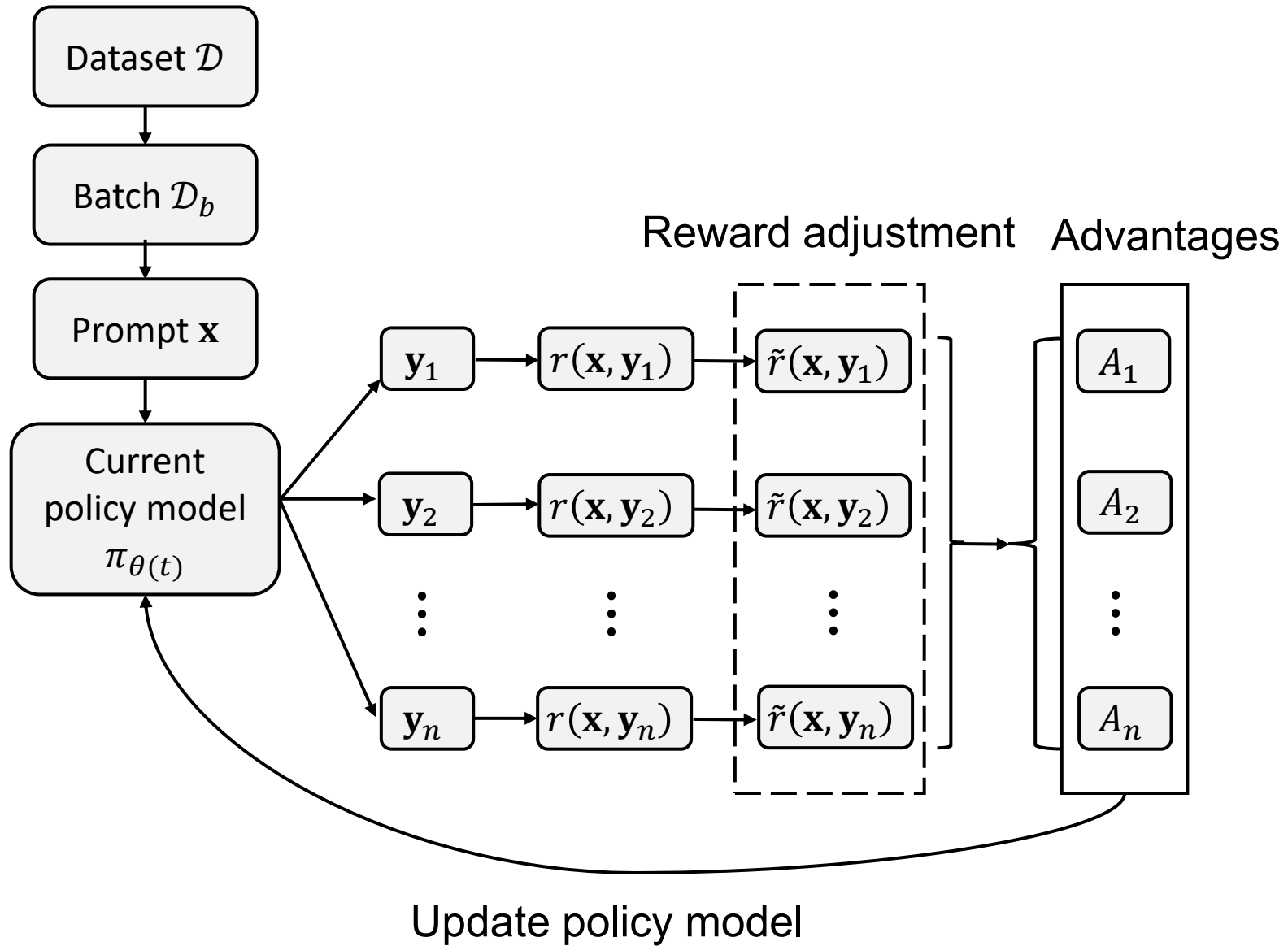
```

Search from
the left side

Search from
the right side

- Integrate the reward adjustment model into the standard GRPO algorithm, resulting in the GRPOVI algorithm.
- The responses are generated from **the current policy model**.
- The corresponding probabilities are from **the initial policy model**.
- The guarantee for reward variance increase can be easily derived.

Corollary 1. Let $\mathbf{x} \in \mathcal{D}$ be any given prompt. Assume that the responses $\{\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n\}$ are generated from the policy model $\pi_{\theta(t)}(\cdot | \mathbf{x})$. If the reward $(\tilde{r}(\mathbf{x}, \mathbf{y}_1), \tilde{r}(\mathbf{x}, \mathbf{y}_2), \dots, \tilde{r}(\mathbf{x}, \mathbf{y}_n))$ is a global optimal solution of the reward adjustment model, with $r_i = r(\mathbf{x}, \mathbf{y}_i)$ and $p_i = \pi_{\theta(0)}(\mathbf{y}_i | \mathbf{x}) \forall i \in \{1, 2, \dots, n\}$, then the reward variance of policy model $\pi_{\theta(0)}(\cdot | \mathbf{x})$ over the response space can be increased for the prompt $\mathbf{x} \in \mathcal{D}$.



- The pseudo code of GRPOVI algorithm.

Algorithm 3 GRPOVI: GRPO with reward variance increase

- 1: **Input:** reference model π_{ref} , reward model r , dataset $\mathcal{D}$, group size n .
 - 2: **Output:** policy model $\pi_{\theta(T)}$.
 - 3: **Initialization:** $\pi_{\theta(0)} = \pi_{\text{ref}}$.
 - 4: **for** step $t = 0, 1, \dots, T$ **do**
 - 5: Choose a batch $\mathcal{D}_b$ from $\mathcal{D}$.
 - 6: Generate n responses $\{\mathbf{y}_i\}_{i=1}^n$ from $\pi_{\theta(t)}(\cdot|\mathbf{x})$ for each $\mathbf{x} \in \mathcal{D}_b$.
 - 7: Compute the probability $p_i = \pi_{\text{ref}}(\mathbf{y}_i|\mathbf{x})$ based on reference model π_{ref} for each $i = 1, 2, \dots, n$.
 - 8: Normalize the probabilities $p_i = \frac{p_i}{\sum_{i=1}^n p_i}$.
 - 9: Compute reward $r_i = r(\mathbf{x}, \mathbf{y}_i)$ for each $i = 1, 2, \dots, n$.
 - 10: Sort $\{p_i\}_{i=1}^n$ and $\{r_i\}_{i=1}^n$ such that $r_1 \geq r_2 \geq \dots \geq r_n$.
 - 11: Compute the adjusted rewards $\{\tilde{r}_i\}_{i=1}^n$ based on $\{p_i\}_{i=1}^n$ and $\{r_i\}_{i=1}^n$ (algorithm [1](#) or algorithm [2](#)).
 - 12: Compute advantages in GRPO based on $\{\tilde{r}_i\}_{i=1}^n$.
 - 13: Update policy model $\pi_{\theta(t)}$ according to GRPO-based RLHF training method.
 - 14: **end for**
 - 15: **return** policy model $\pi_{\theta(T)}$
-

Current policy model

Initial policy model (reference policy model)

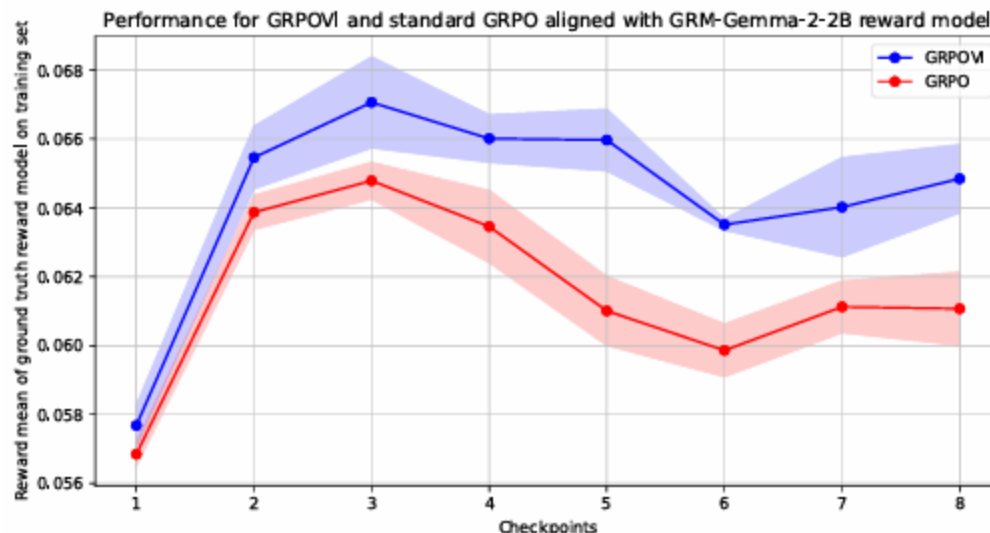
Experiments

- The comparative experiment on the search algorithms.

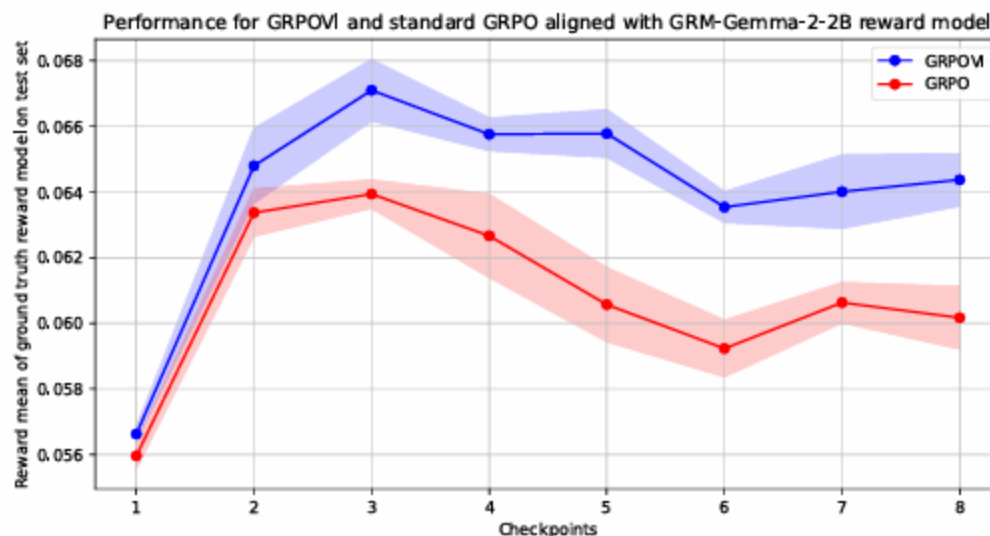
Size n	Enumeration search algorithm		One-pass search algorithm	
	Optimal value f_1^*	Running time (s)	Optimal value f_2^*	Running time (s)
10	0.3918	0.0022	0.3918	0.0015
50	0.4260	0.0438	0.4260	0.0057
100	0.5159	0.1707	0.5159	0.0112
500	0.4970	4.2043	0.4970	0.0539
1000	0.4938	16.9763	0.4938	0.1106
5000	0.4919	408.9071	0.4919	0.5515
10000	0.4933	1659.4376	0.4933	1.0561

Table 1 Comparison of enumeration search algorithm (algorithm 1) and one-pass search algorithm (algorithm 2) for solving (3).

- The experiment between original GRPO and GRPOVI algorithms on LLM RLHF training.
- Policy model: Pythia-1B
- Reward model: GRM-Gemma-2-2B

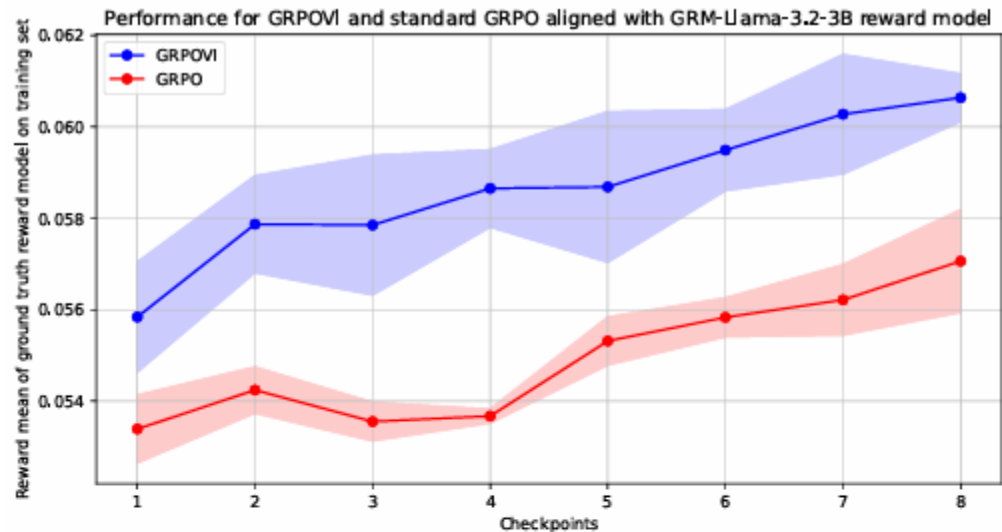


(a) Performance on the training set.

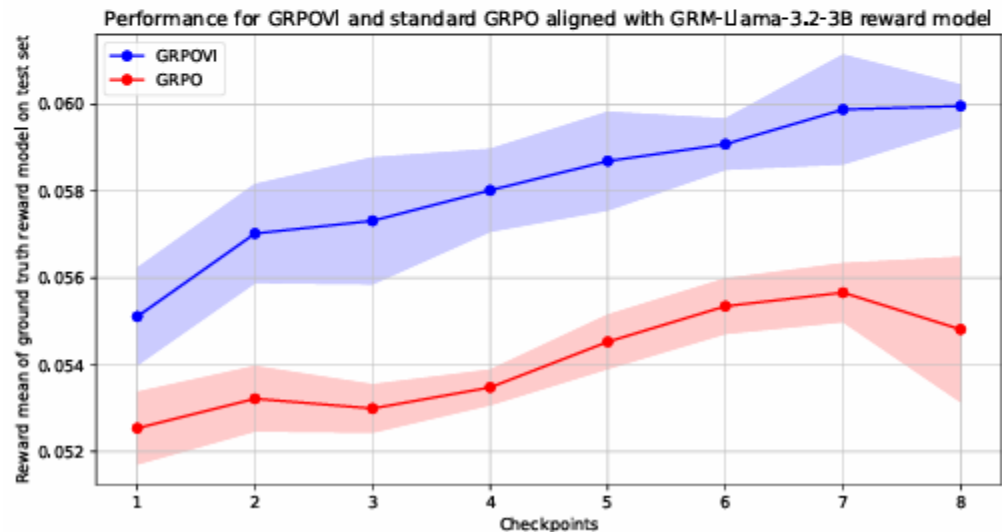


(b) Performance on the test set.

- The experiment between original GRPO and GRPOVI algorithms on LLM RLHF training.
- Policy model: Pythia-1B
- Reward model: GRM-Llama-3.2-3B



(a) Performance on the training set.



(b) Performance on the test set.

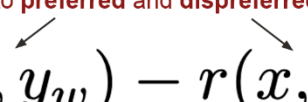
Direct Preference Optimization

Feedback comes as **preferences over model samples**: $\mathcal{D} = \{x^i, y_w^i, y_l^i\}$



Bradley-Terry Model connects rewards to preferences:

Reward assigned to **preferred** and **dispreferred** responses

$$p(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$


Train the reward model by **minimizing negative log likelihood**:

$$\mathcal{L}_R(\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

Now we have a **reward model** r_ϕ that represents* **goodness according to humans**

Now, learn a policy π_θ achieving **high reward** while **staying close** to original model π_{ref}

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)]$$

Sample from policy

Want high reward...

...but keep KL to original model small!

You Can Get Your Optimal Policy with Math



$$\begin{aligned} & \max_{\pi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi} [r(x, y)] - \beta \mathbb{D}_{\text{KL}}[\pi(y|x) \parallel \pi_{\text{ref}}(y|x)] \quad (1) \\ & \hspace{15em} D_{KL}[\pi(y|x) \parallel \pi_{\text{ref}}(y|x)] = \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] \\ & = \max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] \quad (2) \\ & = \min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x, y) \right] \quad (3) \quad \text{Divided by } -\frac{1}{\beta} \\ & \hspace{15em} \log \exp \left(-\frac{1}{\beta} r(x, y) \right) \\ & = \min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right)} \right] \quad (4) \end{aligned}$$

You Can Get Your Optimal Policy with Math



Not a probability distribution!

$$\min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \right] \longrightarrow D_{KL} \left[\pi(y|x) \parallel \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right) \right] \quad (3)$$

$$\frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

$$\text{Where } Z(x) = \sum_u \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

$$= \min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} - \log Z(x) \right] \quad (4)$$

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right),$$

$$Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$



$$r(x, y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

Therefore,

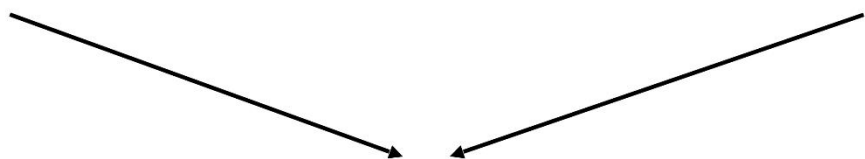
$$\begin{aligned} & r(x, y_2) - r(x, y_1) \\ &= \beta \log \frac{\pi_r(y_2|x)}{\pi_{\text{ref}}(y_2|x)} + \beta \log Z(x) - (\beta \log \frac{\pi_r(y_1|x)}{\pi_{\text{ref}}(y_1|x)} + \beta \log Z(x)) \\ &= \beta \log \frac{\pi_r(y_2|x)}{\pi_{\text{ref}}(y_2|x)} - \beta \log \frac{\pi_r(y_1|x)}{\pi_{\text{ref}}(y_1|x)} \end{aligned}$$

Why Fantastic? We Can Get Rid of Reward



$$\frac{e^A}{e^A + e^B} = \frac{1}{1 + e^{B-A}} = \sigma(B - A)$$

$$r(x, y_2) - r(x, y_1) = \beta \log \frac{\pi_r(y_2|x)}{\pi_{ref}(y_2|x)} - \beta \log \frac{\pi_r(y_1|x)}{\pi_{ref}(y_1|x)}$$


$$p^*(y_1 \succ y_2 \mid x) = \frac{1}{1 + \exp \left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{ref}(y_2|x)} - \beta \log \frac{\pi^*(y_1|x)}{\pi_{ref}(y_1|x)} \right)}$$

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

$$\frac{\sigma'(u)}{\sigma(u)} = \frac{\sigma(u) * (1 - \sigma(u))}{\sigma(u)} = \sigma(-u)$$

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\frac{\sigma'(u)}{\sigma(u)} \nabla_{\theta}(u) \right]$$

$$u = \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} - \beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)}$$



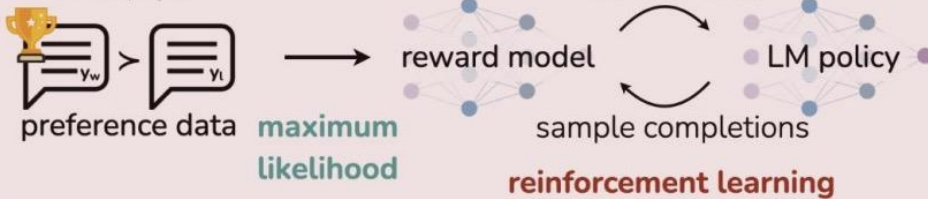
$$\begin{aligned} \nabla_{\theta} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = \\ -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\beta \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \left[\nabla_{\theta} \log \pi(y_w | x) - \nabla_{\theta} \log \pi(y_l | x) \right] \right] \end{aligned}$$

DPO Performs Better



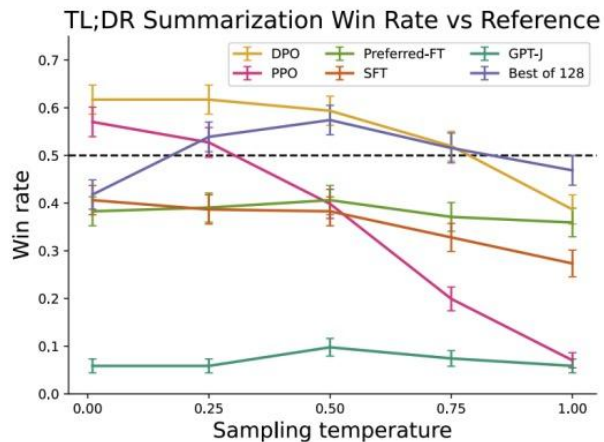
Reinforcement Learning from Human Feedback (RLHF)

x: "write me a poem about
the history of jazz"

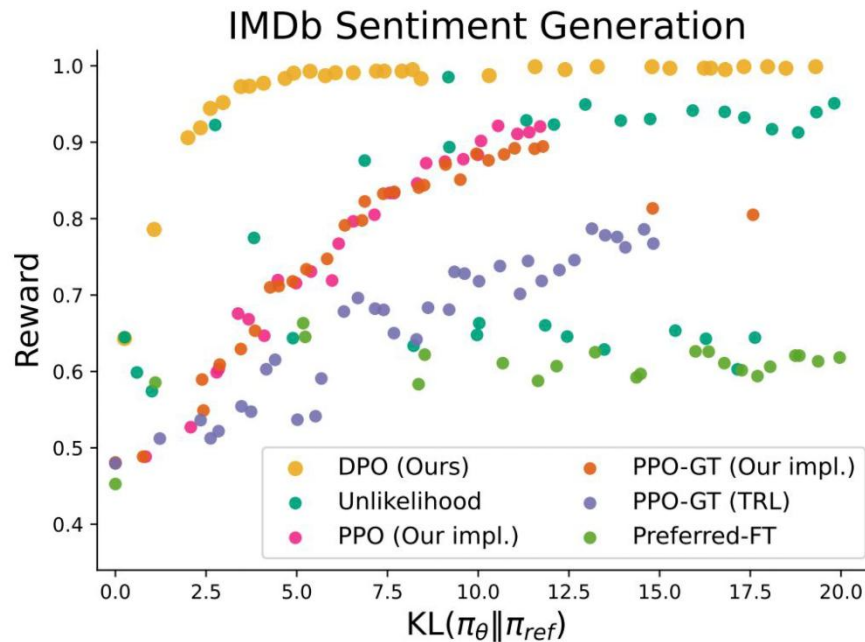


Direct Preference Optimization (DPO)

x: "write me a poem about
the history of jazz"



- You can replace the complex RL part with a very simple weighted MLE objective
- Other variants (KTO, IPO) now emerging too
- TL;DR summarization win rates vs. human-written summaries (GPT-4 as a judge)



1. Generate positive IMDB reviews from GPT2-XL
2. Use pre-trained sentiment classifier as Gold RM
3. Create preferences based on Gold RM
4. Optimize with PPO and DPO

Mistral

4 Instruction Fine-tuning

We train Mistral – Instruct using supervised fine-tuning (SFT) on an instruction dataset followed by Direct Preference Optimization (DPO) [25] on a paired feedback dataset. Mistral – Instruct reaches a score of 8.30 on MT-Bench [33] (see Table 2), making it the best open-weights model as of December 2023. Independent human evaluation conducted by LMSys is reported in Figure 6³ and shows that Mistral – Instruct outperforms GPT-3.5-Turbo, Gemini Pro, Claude-2.1, and Llama 2 70B chat.

Model	🏆 Arena Elo rating	📄 MT-bench (score)	License
GPT-4-Turbo	1243	9.32	Proprietary
GPT-4-0314	1192	8.96	Proprietary
GPT-4-0613	1158	9.18	Proprietary
Claude-1	1149	7.9	Proprietary
Claude-2.0	1131	8.06	Proprietary
Mistral-8x7B-Instruct-v0.1	1121	8.3	Apache 2.0
Claude-2.1	1117	8.18	Proprietary
GPT-3.5-Turbo-0613	1117	8.39	Proprietary
Gemini Pro	1111		Proprietary
Claude-Instant-1	1110	7.85	Proprietary
Tulu-2-DPO-70B	1110	7.89	AI2 ImpACT Low-risk
Yi-340-Chat	1110		Yi License
GPT-3.5-Turbo-0314	1105	7.94	Proprietary
Llama-2-70b-chat	1077	6.86	Llama 2 Community

Figure 6: LMSys Leaderboard. (Screenshot from Dec 22, 2023) Mistral 8x7B Instruct v0.1 achieves an Arena Elo rating of 1121 outperforming Claude-2.1 (1117), all versions of GPT-3.5-Turbo (1117 best), Gemini Pro (1111), and Llama-2-70b-chat (1077). Mistral is currently the best open-weights model by a large margin.

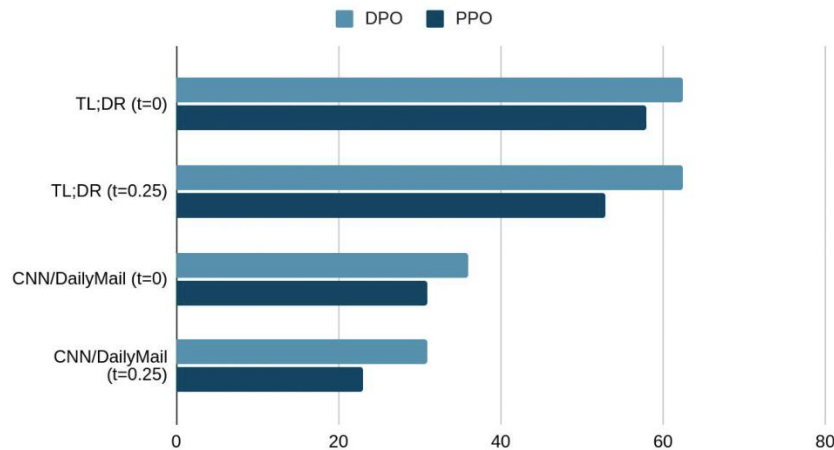
LLaMa3

Instruction fine-tuning

To fully unlock the potential of our pretrained models in chat use cases, we innovated on our approach to instruction-tuning as well. Our approach to post-training is a combination of supervised fine-tuning (SFT), rejection sampling, proximal policy optimization (PPO), and direct preference optimization (DPO). The quality of the prompts that are used in SFT and the preference rankings that are used in PPO and DPO has an outsized influence on the performance of aligned models. Some of our biggest improvements in model quality came from carefully curating this data and performing multiple rounds of quality assurance on annotations provided by human annotators.

Learning from preference rankings via PPO and **DPO** also greatly improved the performance of Llama 3 on reasoning and coding tasks. We found that if you ask a model a reasoning question that it struggles to answer, the model will sometimes produce the right reasoning trace: The model knows how to produce the right answer, but it does not know how to select it. Training on preference rankings enables the model to learn how to select it.

Win Rates



1. DPO is trained only on the Reddit TL;DR feedback data.
2. PPO uses a trained reward function and additional prompts for RL training.
3. We evaluate the trained policies on OOD CNN/DailyMail news summarization task.

DPO fits an implicit reward function:

1. **Is the DPO implicit reward as good as the explicit one?**
2. Does using a weaker optimizer, such as PPO provide a better solution (regularization).

RewardBench: Evaluating Reward Models

Evaluating the capabilities, safety, and pitfalls of reward models

[Code](#) | [Eval. Dataset](#) | [Prior Test Sets](#) | [Results](#) | [Paper](#) | Total models: 74



 **RewardBench Leaderboard**
 RewardBench - Detailed
 [Prior Test Sets](#)
[About](#)
[Dataset Viewer](#)

Model Search (delimit with ,)

☒ Seq. Classifiers

☒ DPO

☒ Custom Classifiers

☐ Generative

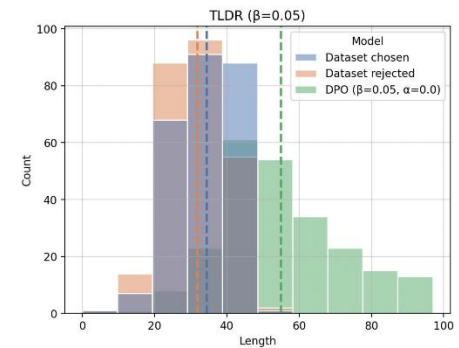
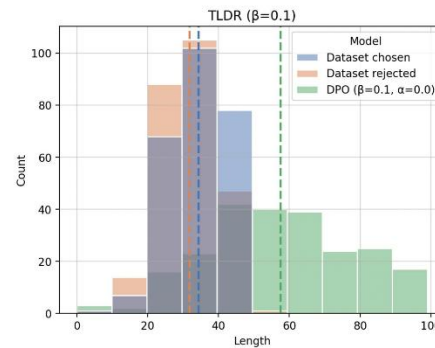
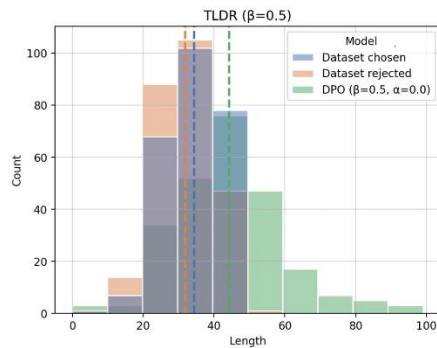
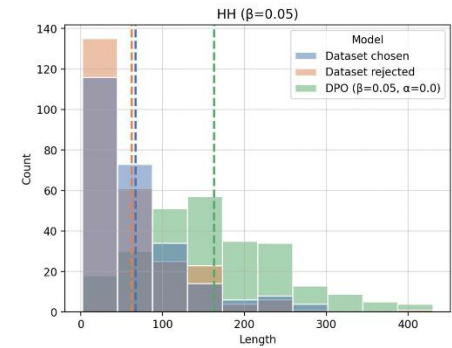
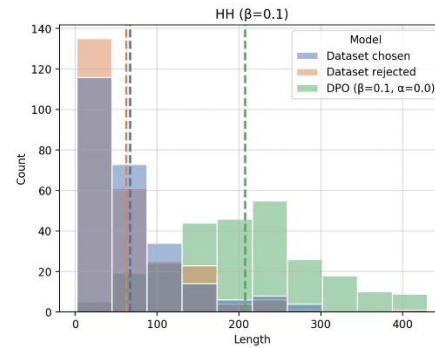
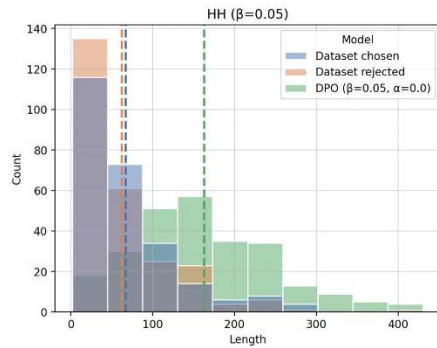
☐ AI2 Experiments

▲	Model	▲	Model Type	▲	Score	▲	Chat	▲	Chat Hard	▼	Safety	▲	Reasoning	▲	Prior Sets (0.5 weight)	▲
24	Qwen/Qwen1.5-14B-Chat		DPO		69.76		57.3		70.2		76.3		89.6		41.2	
26	Qwen/Qwen1.5-7B-Chat		DPO		68.75		53.6		69.1		74.8		90.4		42.9	
12	upstage/SOLAR-10.7B-Instruct-v1.0		DPO		73.99		81.6		68.6		85.5		72.5		49.5	
29	Qwen/Qwen1.5-72B-Chat		DPO		68.21		62.3		66		72		85.5		42.3	
3	openbmb/Eurus-RM-7b		Seq. Classifier		81.55		98		65.6		81.2		86.3		71.7	
1	Cohere March 2024		Custom Classifier		85.69		94.7		65.1		90.3		98.2		74.6	
2	sfairXC/FsfairX-LLaMA3-RM-v0.1		Seq. Classifier		83.62		99.4		65.1		87.8		86.4		74.9	
11	mistralai/Mixtral-8x7B-Instruct-v0.1		DPO		74.74		95		64		73.4		78.7		50.3	
33	Qwen/Qwen1.5-MoE-A2.7B-Chat		DPO		67.54		72.9		63.2		67.8		77.4		45.4	
49	Qwen/Qwen1.5-0.5B-Chat		DPO		55.01		35.5		62.9		66.1		59.8		46.3	
17	HuggingFaceH4/zephyr-7b-beta		DPO		71.77		95.3		62.7		61		77.9		52.2	
48	Qwen/Qwen1.5-4B-Chat		DPO		56.14		38.8		62.7		61.8		66.9		44.7	
13	HuggingFaceH4/zephyr-7b-alpha		DPO		73.42		91.6		62.5		74.3		75.1		53.5	
15	0-hero/Matter-0.1-7B-boost-DPO-preview		DPO		73.35		91.1		61		66.3		83.9		55.7	

DPO fits an implicit reward function:

1. Is the DPO implicit reward as good as the explicit one?
2. **Does using a weaker optimizer, such as PPO provide a better solution (regularization).**

DPO vs PPO: Reward Hacking



Disentangling Length from Quality in Direct Preference Optimization, Park et. al.

DPO fits an implicit reward function:

1. Is the DPO implicit reward as good as the explicit one? **YES**
2. **Does using a weaker optimizer, such as PPO provide a better solution (regularization).**

NO